

Security

Operating Systems – EDA093/DIT401

Vincenzo Gulisano
vincenzo.gulisano@chalmers.se



CHALMERS
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF
GOTHENBURG

Reading instructions

- Chapter 9, from 9.1 to 9.5.1 (included)

Agenda

- Introduction/Overview
- Protection Domains
- Access Control Lists
- Capabilities
- Authentication
- Exploiting software bugs: buffer overflow attacks

Agenda

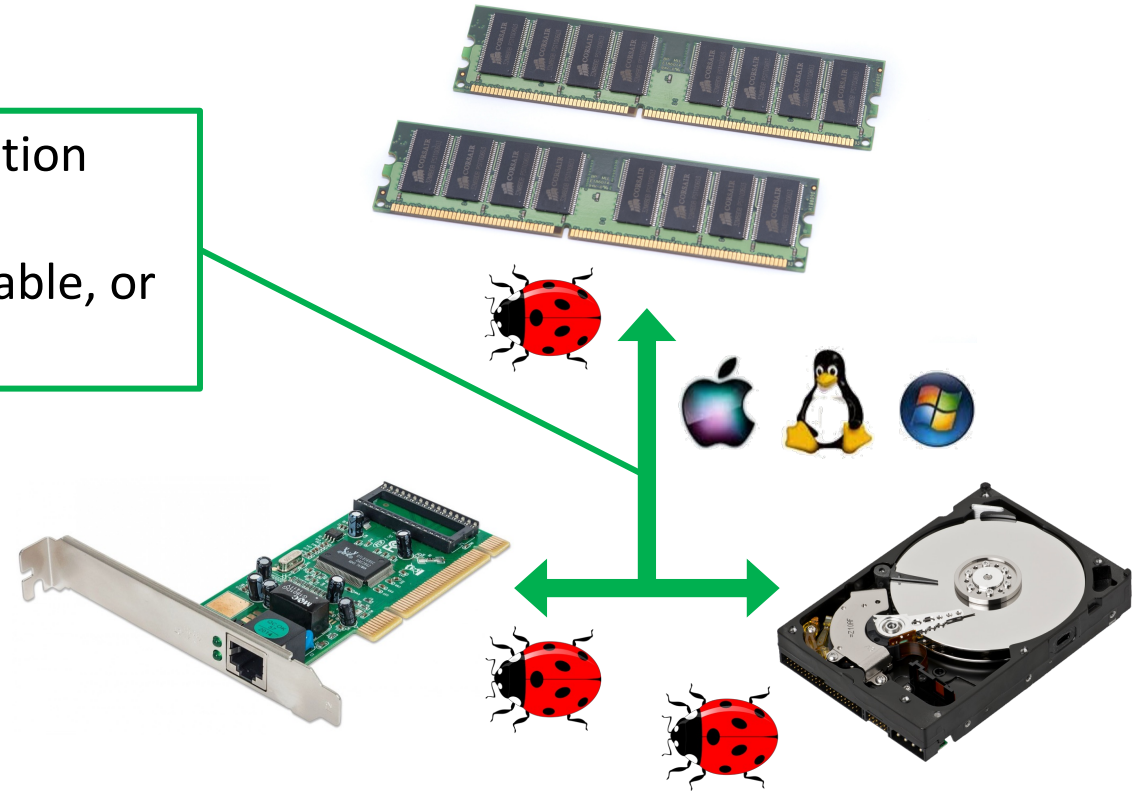
- Introduction/Overview
- Protection Domains
- Access Control Lists
- Capabilities
- Authentication
- Exploiting software bugs: buffer overflow attacks

Important information
may be private,
commercially valuable, or
legally protected.

Attacker might want to:

- read
- modify/delete
- prevent the owner from accessing it
- ...

bugs are a way to reach such goals
Vulnerability: a weakness that can be used to violate a security policy.
Exploit: code, input, or a technique that takes advantage of a vulnerability.





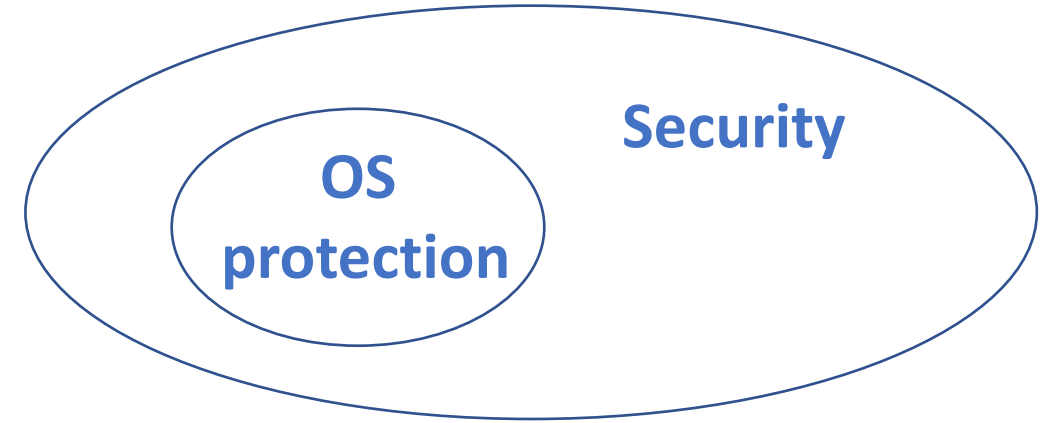
Features

An email service that supports images and file attachments

Simplicity and security

An email service restricted to plain ASCII text

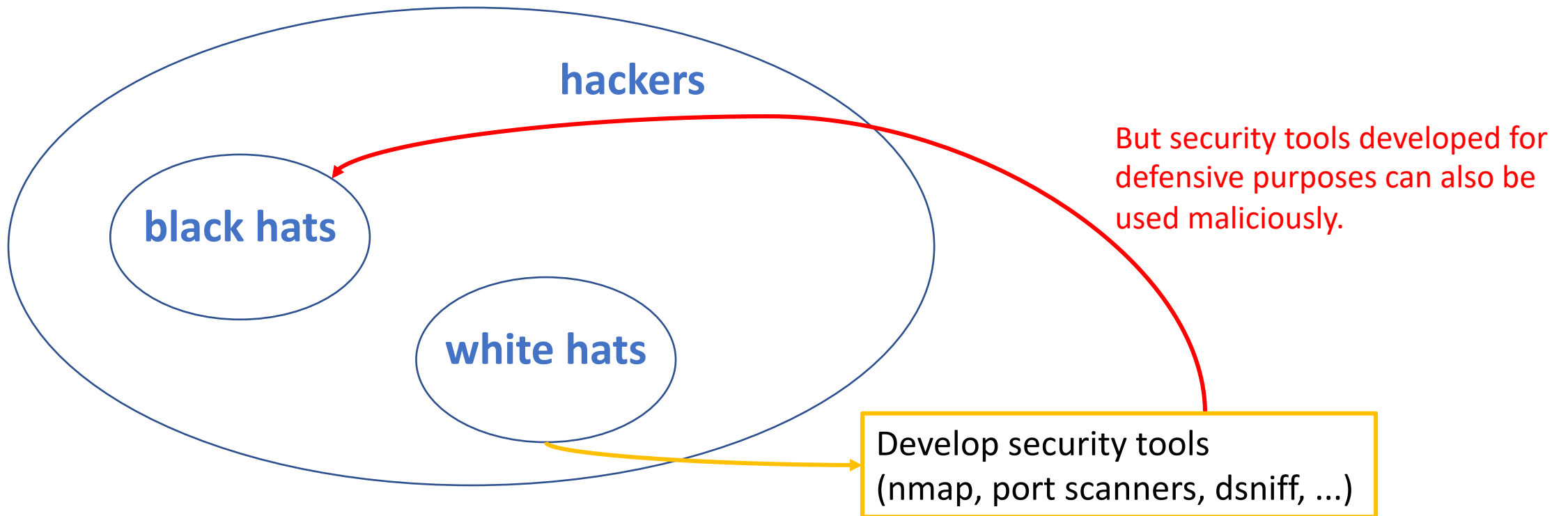
It's complicated...



- Attacks
 - Worm: freestanding program, propagates without user intervention
 - Virus: attaches to another file/program, propagates through user interaction
 - Trojan horse: malware disguised as legitimate software
 - DoS, DDoS, ...
- Being the victim of a security incident does not imply that the attacker is specifically targeting you or your data.
- This lecture focuses on attacks and protection mechanisms related to the OS.

Goal	Threat
Confidentiality	Exposure of data
Integrity	Tampering with data
Availability	Denial of service

There are, of course, additional security goals, including authenticity, accountability, non-repudiation, and privacy.

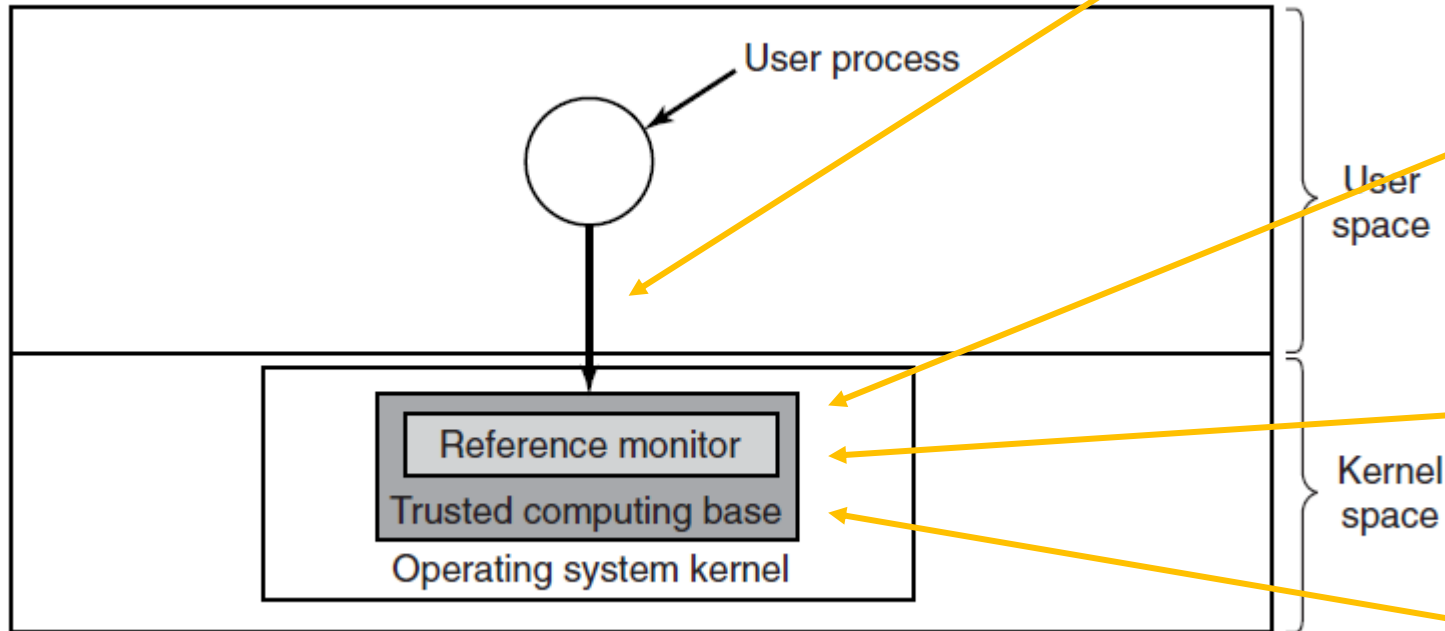


Attacks and protection mechanisms

- Attacks
 - Passive (e.g., sniff network traffic and try to break encryption)
 - Active (take control of a browser and use it to execute code)
- Protection mechanisms
 - Cryptography
 - Software hardening
 - And more... (e.g., Intel SGX provides enclave instructions that isolate selected code and data from other software, including privileged system software, subject to SGX's threat model and limitations)

Trusted Computing Base (TCB)

A reference monitor must mediate every security-relevant access to protected objects. System calls are common entry points, but not every system call necessarily performs an access-control decision.



The TCB consists of all hardware and software components whose failure could violate the security policy. It commonly includes security-relevant hardware, the kernel or its security-critical subset, and trusted privileged services.

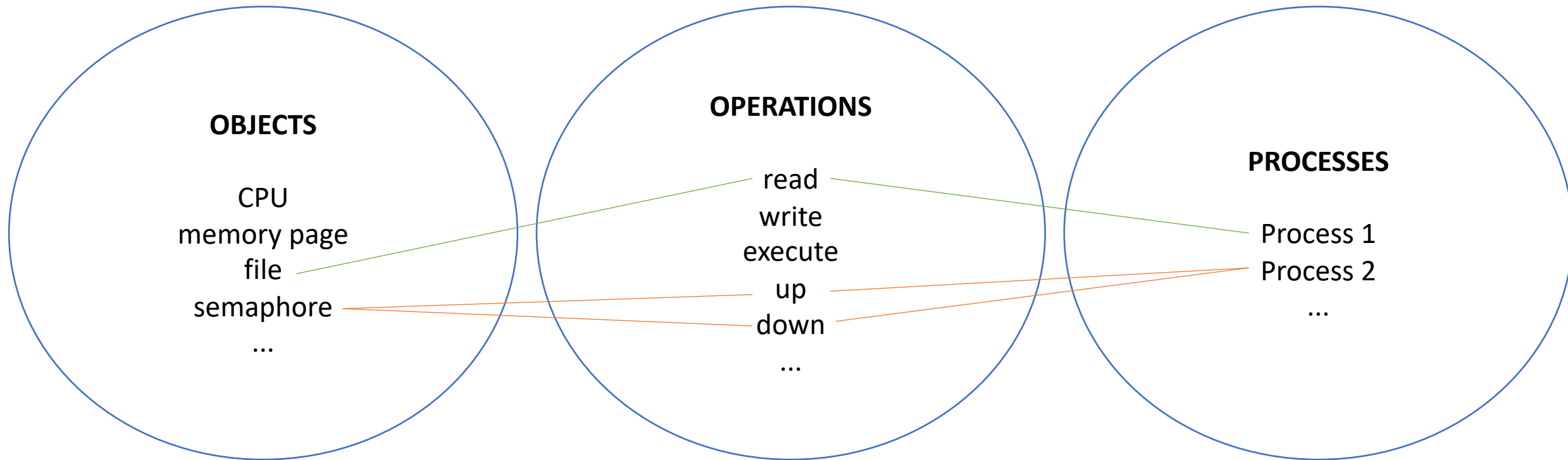
The TCB should be as small as practical.

The security model must define what is protected, which subjects may perform which operations, and the assumed power of the adversary.

Agenda

- Introduction/Overview
- **Protection Domains**
- Access Control Lists
- Capabilities
- Authentication
- Exploiting software bugs: buffer overflow attacks

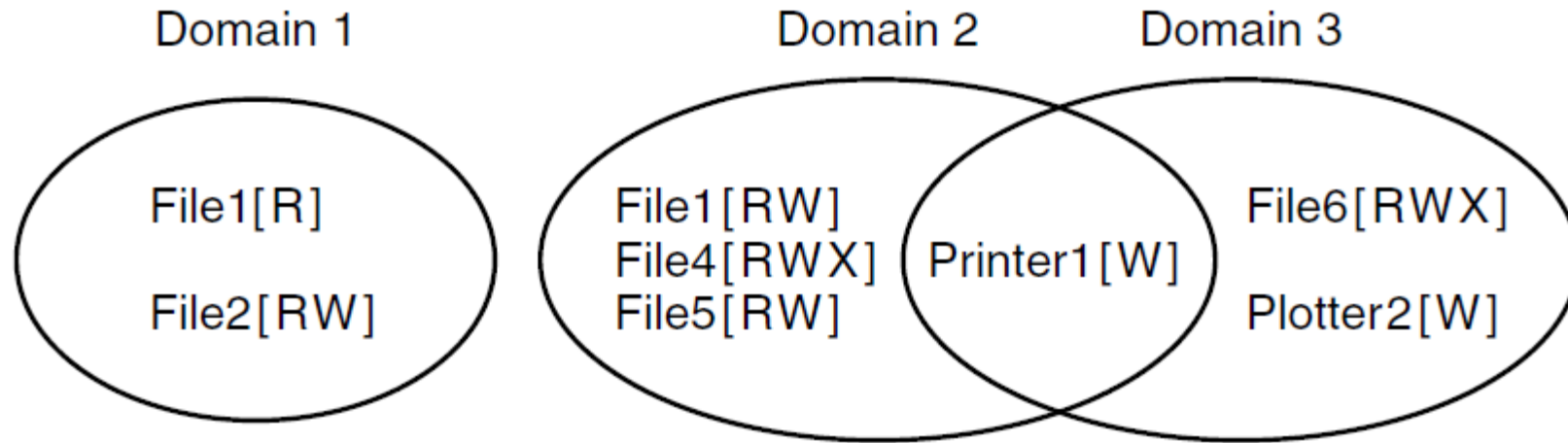
Protection domains



Protection domains

- Domain: set of <object, rights> pairs
- For the simplified examples in this lecture, one domain corresponds to one user. More generally, a domain is any set of access rights associated with an execution context.
- Security works best applying the Principle of Least Authority: give each domain only the objects and rights required to perform its task.

Protection domains



- At each point in time, each process runs in some protection domain
- Some systems permit a process to switch domains through controlled operations.
- Unix: each process has UID and GID

Domain	Object							
	File1	File2	File3	File4	File5	File6	Printer1	Plotter2
1	Read	Read Write						
2			Read	Read Write Execute	Read Write		Write	
3						Read Write Execute	Write	Write

Figure 9-4. A protection matrix.

Access Control Lists

Domain	Object										
	File1	File2	File3	File4	File5	File6	Printer1	Plotter2	Domain1	Domain2	Domain3
1	Read	Read Write								Enter	
2			Read	Read Write Execute	Read Write		Write				
3						Read Write Execute	Write	Write			

Capability list

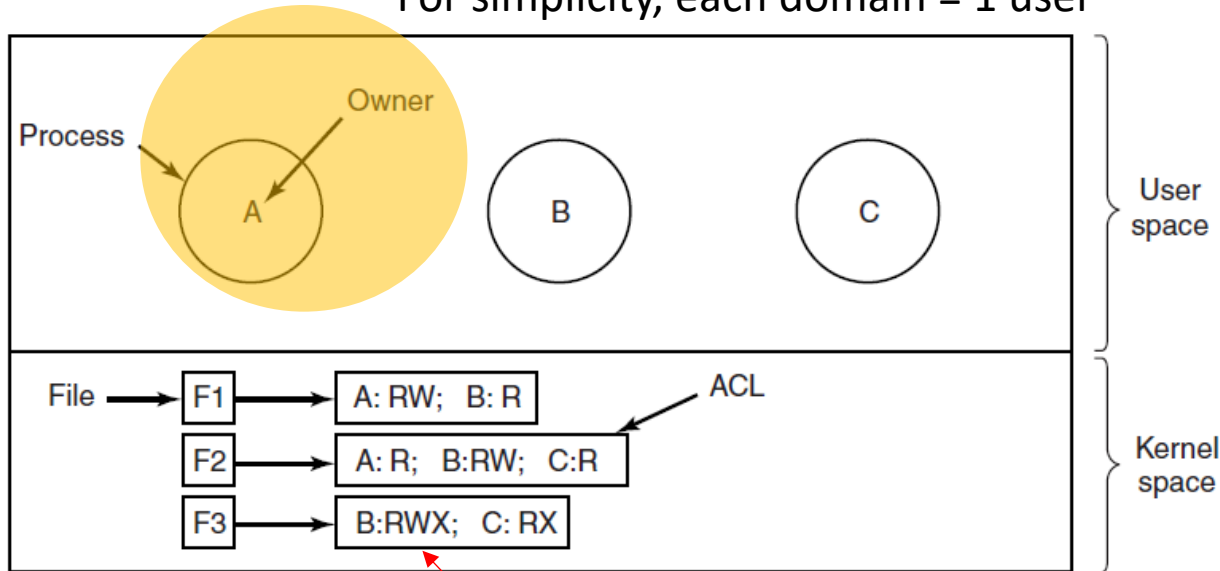
Figure 9-5. A protection matrix with domains as objects.

Agenda

- Introduction/Overview
- Protection Domains
- **Access Control Lists**
- Capabilities
- Authentication
- Exploiting software bugs: buffer overflow attacks

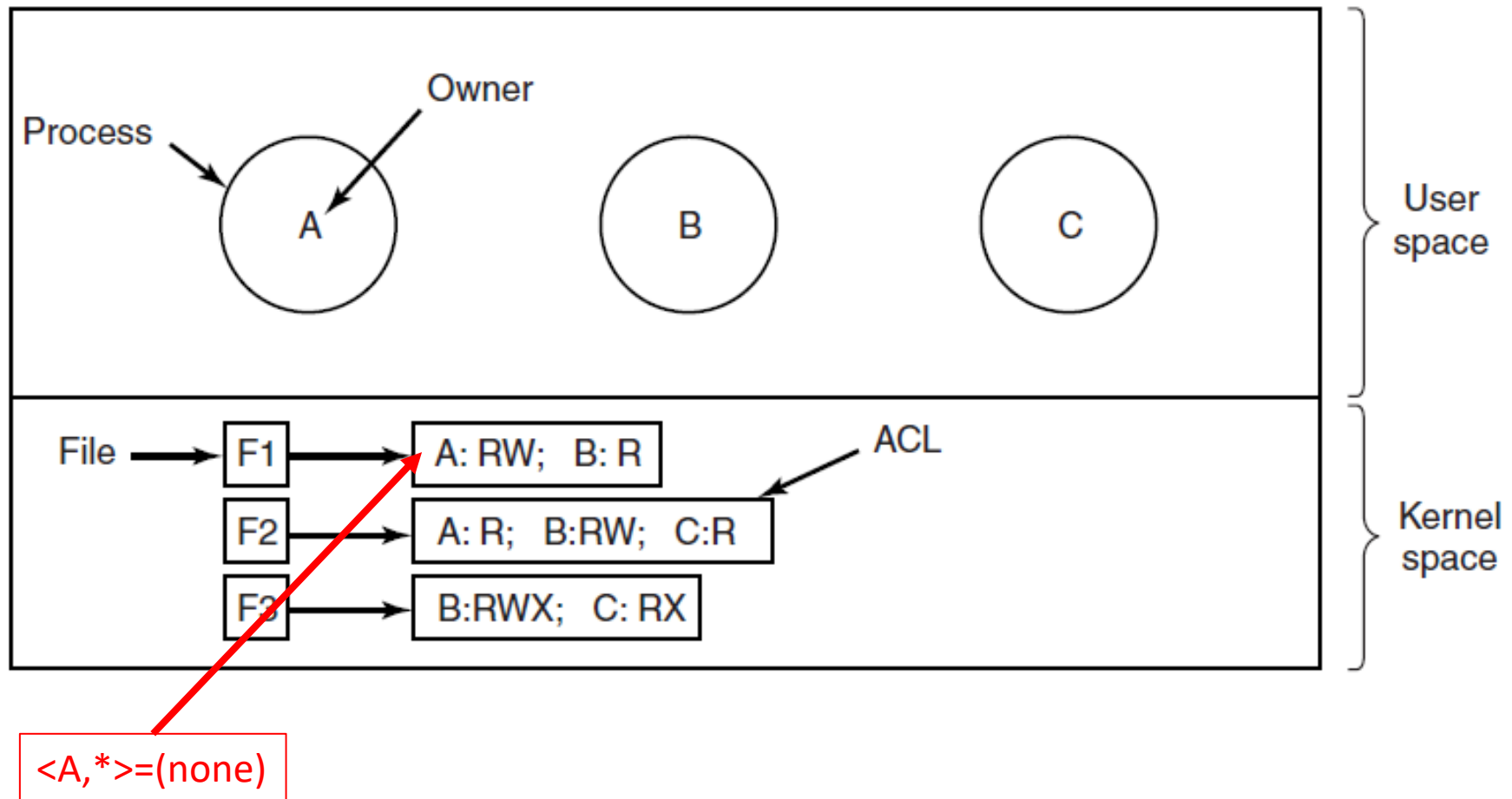
Access Control Lists

For simplicity, each domain = 1 user



In the simplified ordered-ACL model used in these slides, entries are scanned in order, and the first matching entry determines the rights.

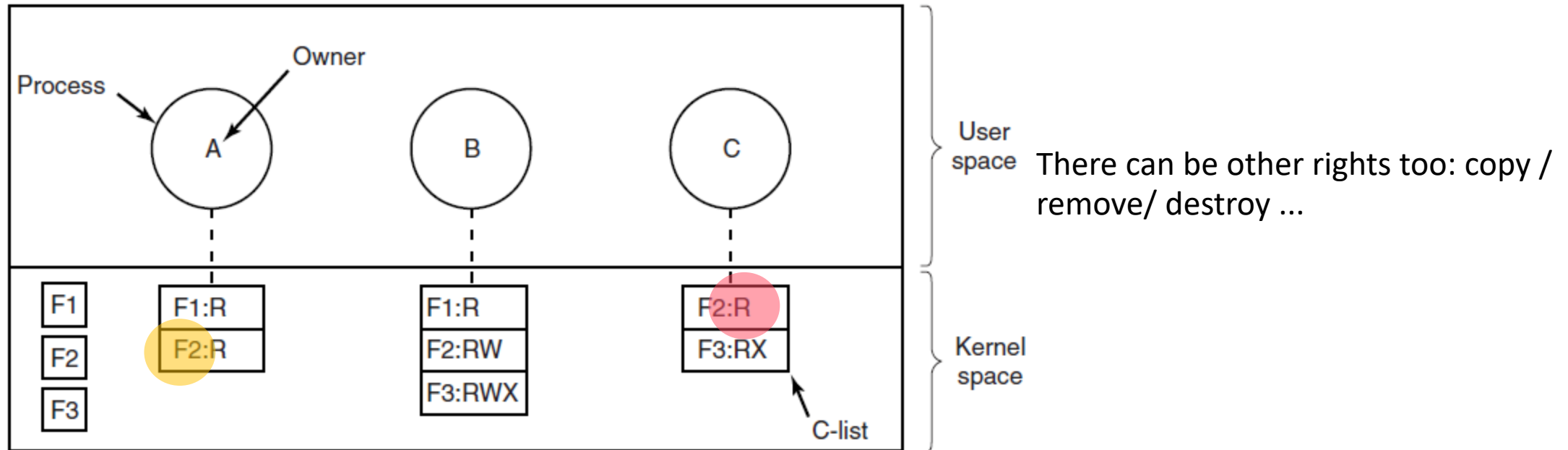
- RWX rights are common, but there could be more
- rights might be per group of users, not an individual user
 - Could be stored as pairs (UID1,GID1):... (UID2,GID2):... – this models *ROLES*
 - Could include wildcards (UID1,*)
 - could be stored individual UID1:... UID2:... GID1:...



Agenda

- Introduction/Overview
- Protection Domains
- Access Control Lists
- **Capabilities**
- Authentication
- Exploiting software bugs: buffer overflow attacks

Capabilities



The object contained in a capability list could be a pointer to another capability list (to facilitate the sharing of subdomains)

Capabilities

- How to prevent a user from tampering/altering capabilities?
 - Tagged architecture: each word in memory has a special bit. If set, word contains a capability, and can only be changed in kernel mode
 - Keep C-lists in OS memory, user processes can point to them
 - Keep in user space and encrypt them so that users cannot modify them



Figure 9-9. A cryptographically protected capability.

If the user alters the Rights, the $f()$ computed by the server / OS and the one in the capability will differ!

Agenda

- Introduction/Overview
- Protection Domains
- Access Control Lists
- Capabilities
- **Authentication**
- Exploiting software bugs: buffer overflow attacks

Authentication

- A secure system requires authentication
 - Need to know who the user is before enforcing a certain policy (which is user-specific)
- Approaches:
 - something the user knows (password)
 - something the user has (a smart card)
 - something the user is (the owner of a certain fingerprint)

About passwords...

- VERY BAD IDEAS:
 - store <user id, password> to check if the password of a user is correct
 - do not have the BIOS password-protected
 - Naïve mechanisms:
 - complain about the login name being wrong
 - show * while users are typing characters
 - weak passwords
 - default login/password → STUXNET!!!!
 - re-used passwords



Unix authentication (alternative 1, not secure)

- passwords are not stored, what is stored is
<user id, result of encrypting a fixed block of data with the user password>
- Easy to attack using dictionary attacks
 - Try all common passwords/combinations of words until one matches the encrypted set of characters associated with a user id → password found

Unix authentication + n-bit salt

Bobbie, 4238, e(Dog, 4238)
Tony, 2918, e(6%%TaeFF, 2918)
Laura, 6902, e(Shakespeare, 6902)
Mark, 1694, e(XaB#Bwcz, 1694)
Deborah, 1092, e(LordByron,1092)

- have unique random n-bit prefixes for each user
- Now 1 dictionary is not enough, the attacker needs 2^n dictionaries --> unfeasible

One-time passwords / one-way hash chain

- Use hash function f with input and output having the same length
- Choose a number n (how many passwords)
- $P_1 = \underbrace{f(f(f(s)))}_{n \text{ of them}}$
- $P_2 = \underbrace{f(f(f(s)))}_{n-1 \text{ of them}}$
- $P_i = f(P_{i+1}) \rightarrow$ but you cannot find P_{i+1} from P_i

One-time passwords / one-way hash chain

- Server stores P_0 (which is $f(P_1)$) and value 1
- First login, sends 1 to user, gets P_1 , compares $f(P_1)$ with P_0 and, if successful, stores P_1 and value 2

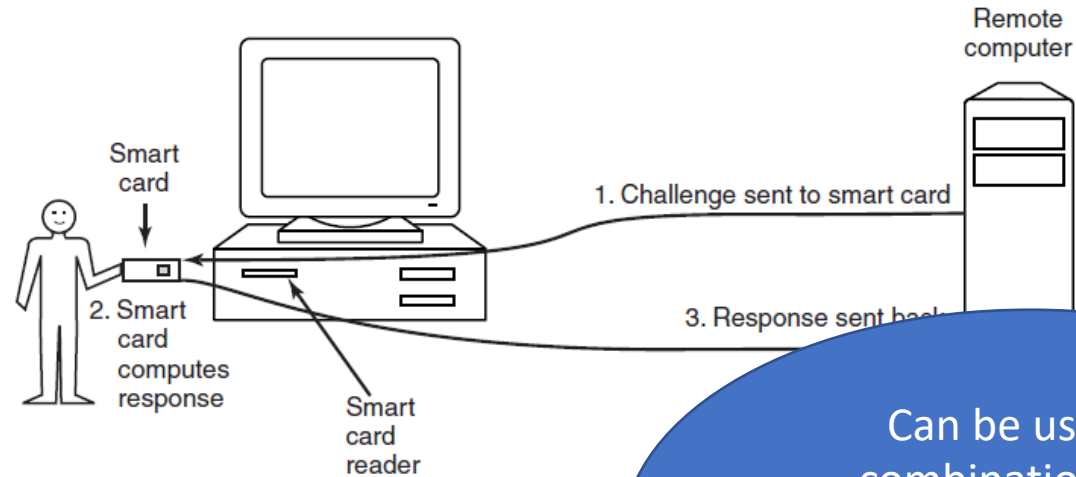
... can be repeated n times

Challenge-Response authentication

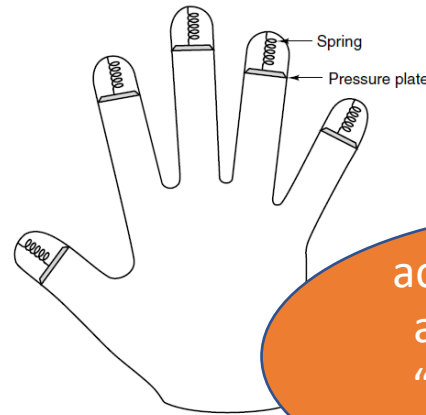
- Chose from a long list of questions
 - The user knows the answer, no need to “write it down somewhere”
 - Storage on server must be secure of course
- Let the user choose an algorithm (e.g., x^2) and ask the user to compute the right answer
- If an initial password/key k can be shared between user and server,
 - server shares r
 - both user and client compute $f(r,k)$ and compare results – f can be known
--> with high computing power → complex f and large r/k → hard for the attacker to break

And more

- Authentication using a physical object



- Biometrics



account for variability
account for enough
“distance” between
users

Can be used in
combination with
passwords, of course
(multiple techniques can
be used at the same time)

About cards

- Magnetic stripe cards
 - read only
 - can contain the user password
- Chip cards
 - stored value cards
 - value in them can be updated
- Smart cards
 - mount a small CPU
 - Can send messages/encrypting them in some cases
 - This can be useful to authorize payments when there's no connectivity
 - Can run challenge-response authentication

Agenda

- Introduction/Overview
- Protection Domains
- Access Control Lists
- Capabilities
- Authentication
- Exploiting software bugs: buffer overflow attacks

Exploiting software

- Cracking passwords is only one way of course to gain control
- Alternatives
 - drive-by-download: infect browser, have user download malicious content, malicious content downloads malware, ...
 - **exploit bugs**
 - OS have many...



Buffer overflow attacks

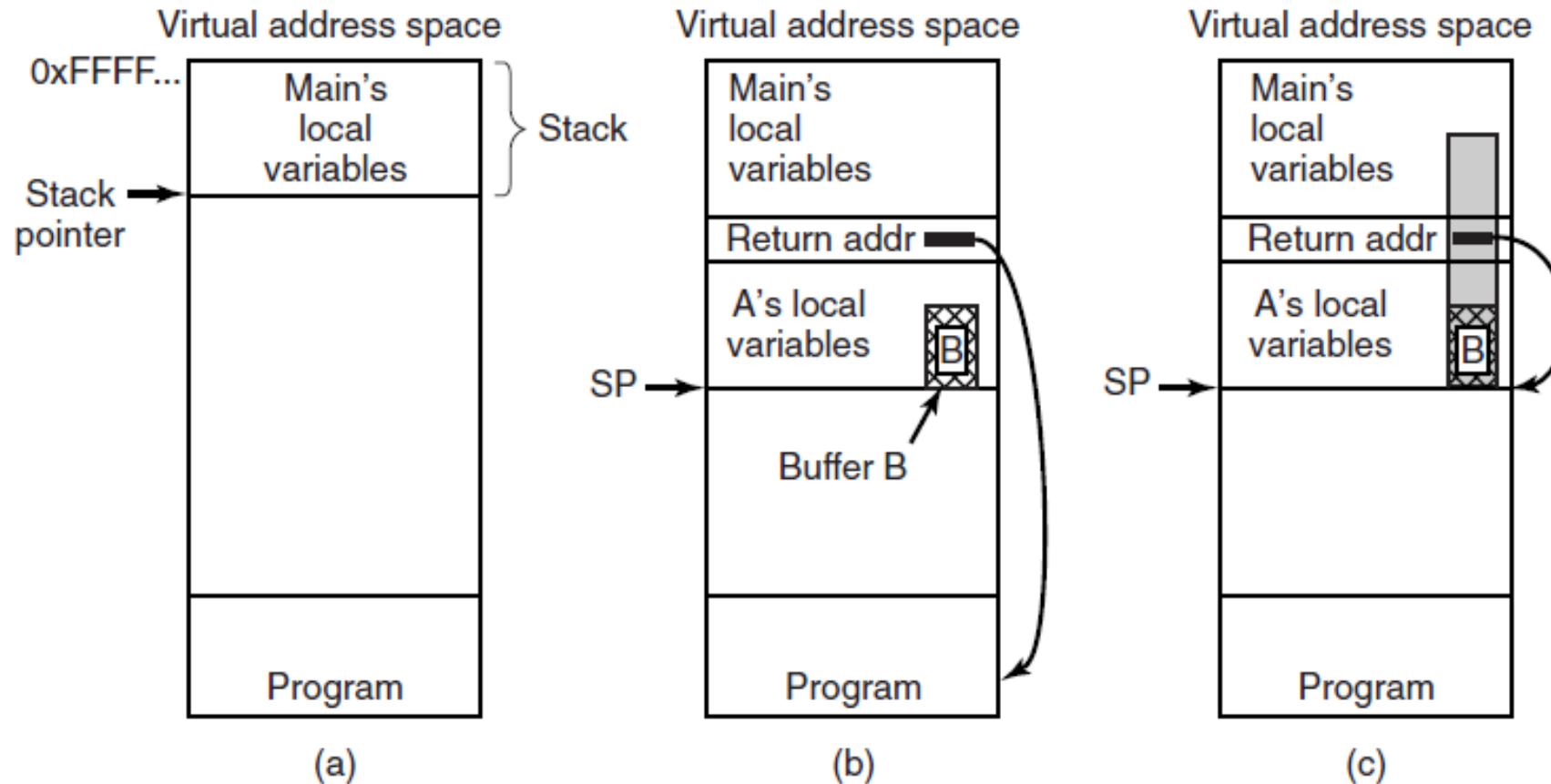
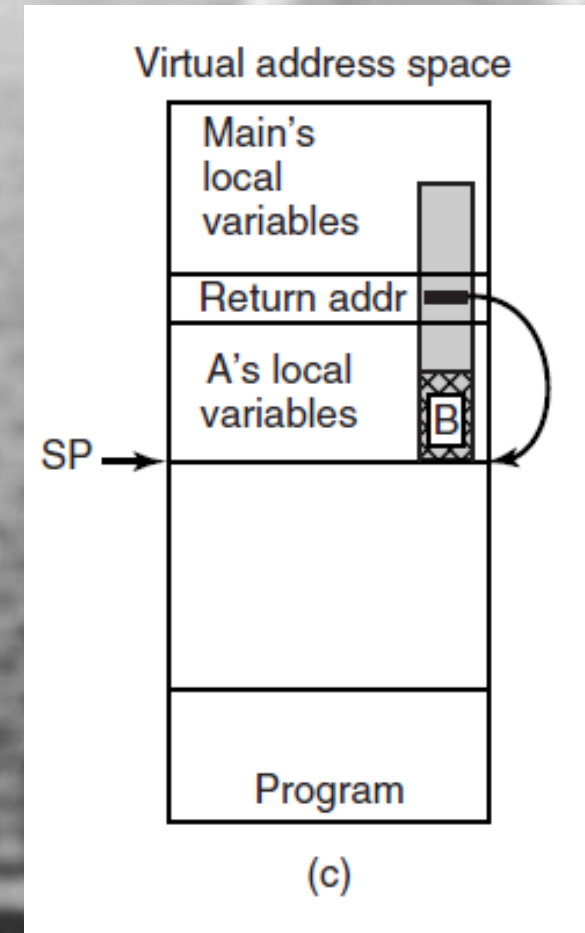


Figure 9-21. (a) Situation when the main program is running. (b) After the procedure A has been called. (c) Buffer overflow shown in gray.

Possible defense: Stack Canaries

- Just below the return address, add random canary value
- When compiling a program, insert code that, upon return from a function, check if the canary value is still correct
- Is it not? → return address has been modified WHP
- NOTICE: stack canaries do not always work, check "Avoiding Stack Canaries" in section 9.7



Alternative defense strategy (not 100% secure either)

- No-execute/NX bit (hardware)
- hardware bit that ensures data segments are readable and modifiable, but not executable
- Only text section can be executed
- Code injection attacks do not *work*

Questions

Consider the following protection matrix. Which statement is correct?

Domain	File 1	File 2	File 3
D1	R	RW	-
D2	-	R	W
D3	RW	-	-

1. Each column corresponds to an access control list, and each row corresponds to a capability list.
2. Each row corresponds to an access control list, and each column corresponds to a capability list.
3. Neither representation corresponds to rows or columns of the matrix.

Consider the following protection matrix

Domain	File 1	File 2	File 3	File 4	Printer 1	Printer 2
D1	R	-	-	-	W	-
D2	RW	R	X	W	-	W
D3	-	-	R	-	-	W
D4	W	R	-	-	-	-

The administrator wants to determine **everything that Domain 2 can access**. Which representation requires inspecting a single list?

1. Access control lists
2. Capability lists
3. Both

Consider the following protection matrix

Domain	File 1	File 2	File 3	File 4	Printer
D1	R	RW	-	X	-
D2	W	-	R	-	RW
D3	RX	-	-	W	R
D4	-	X	RW	R	-

Assume that:

- each nonempty cell is stored as one permission entry;
- groups, wildcards, shared lists, and other compression mechanisms are not used; and
- list metadata is ignored.

Which representation requires fewer permission entries?

1. Access-control lists
2. Capability lists
3. They require the same number

Consider the following protection matrix

Domain	File 1	File 2	File 3	File 4
D1	-	-	R	-
D2	R	-	W	-
D3	-	W	RW	-
D4	-	-	X	R
D5	W	-	RX	-

The administrator wants to find and revoke **all permissions on File 3**.

Which representation requires inspecting and updating a single list?

1. Access control lists
2. Capability lists
3. Both

An access-control list contains the following entries:

$\langle A: R, B: R, C: R, D: RW, *: none \rangle$

Group **G** contains users A, B, C, and D. Which compact access control list is equivalent to the original list for every user?

1. $\langle G: R, D: RW, *: none \rangle$
2. $\langle D: RW, G: R, *: none \rangle$
3. $\langle G: RW, *: none \rangle$

An access-control list contains the following entries:

$\langle (A, *): R, (*, G1): RW, (*, *): \text{none} \rangle$

A process with UID=A and GID=G1 attempts to write to the object. Is the write operation permitted?

1. Yes
2. No
3. There is not enough information

Assume that:

1. User A owns all three files;
2. folder permissions can be ignored; and
3. tool is a binary executable.

Consider the command trace

<code>cat report.txt</code>	succeeds
<code>printf 'x' >> report.txt</code>	fails
<code>printf 'x' >> data.bin</code>	succeeds
<code>./tool</code>	succeeds
<code>cat tool</code>	fails

Which permission setup is consistent with the complete command trace?

Setup	report.txt	data.bin	tool
1	r--	rw-	--x
2	rw-	r--	r-w
3	r--	rw-	r-x